GPU Merge Path

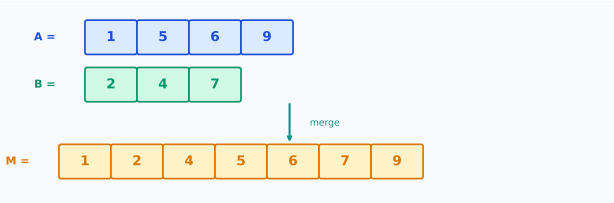
Parallel Merging of Sorted Arrays

Erwan Ouabdesselam · Samy Aimeur

March 30, 2026

GPU Programming · Master MS2A, Sorbonne Université

The problem : merging two sorted arrays



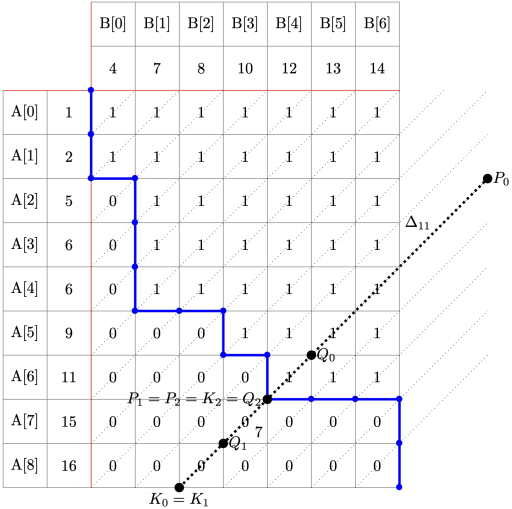
Why is the naive approach hard to parallelize ?

- Each step depends on the previous one
- Cannot compute $M[k]$ without knowing $M[0 \dots k-1]$

Sequential complexity

$\mathcal{O}(|A| + |B|)$ but uses **only one thread**

Intuition : the grid and the Merge Path



Key idea

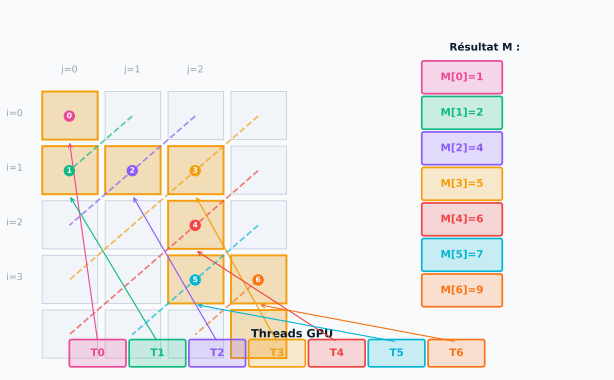
The merge can be represented as a **path in a grid** of size $|A| \times |B|$.

- Cell $(i, j) = 1$ if $B[i] > A[j]$, otherwise 0
- The path follows the **0/1 boundary**

Key observation

Each **diagonal** intersects the path at **exactly one point**.

Question 1 : `mergeSmall_k`



$d = |A| + |B|$ (total merge size)

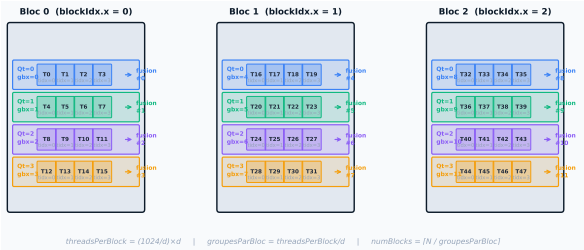
Thread k :

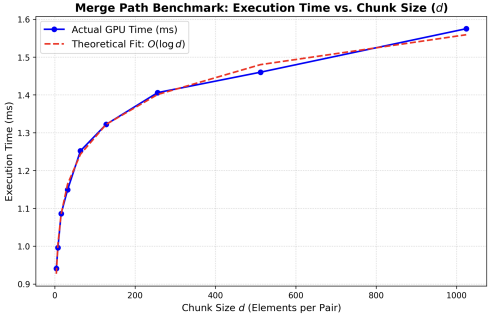
1. Identifies its diagonal $\#k$
2. Finds the intersection using **binary search** in $\mathcal{O}(\log d)$
3. Writes $M[k]$ independently of other threads

Speedup

$\mathcal{O}(\log d)$ per thread
vs $\mathcal{O}(d)$ sequential

Question 2a : `mergeSmallBatch_k`





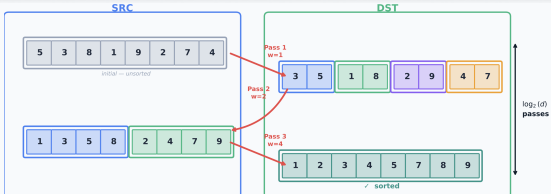
Core idea

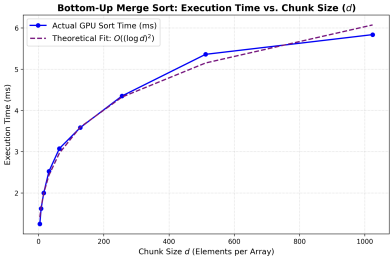
Each block is partitioned into groups of d threads. Each group performs an independent merge in parallel, allowing many small merges to be executed simultaneously and improving GPU utilization.

The 3 indices

- `Qt = threadIdx.x / d`
  local group within the block
- `tidx = threadIdx.x - Qt*d`
  local diagonal within the merge
- `gbx = Qt + blockIdx.x*(tpb/d)`
  global merge index

Question 2b : `sortSmallBatch_k`





Core idea

Parallel bottom-up merge sort on GPU : each group of d threads sorts one array through $\log d$ successive merge passes.

Shared memory + ping-pong

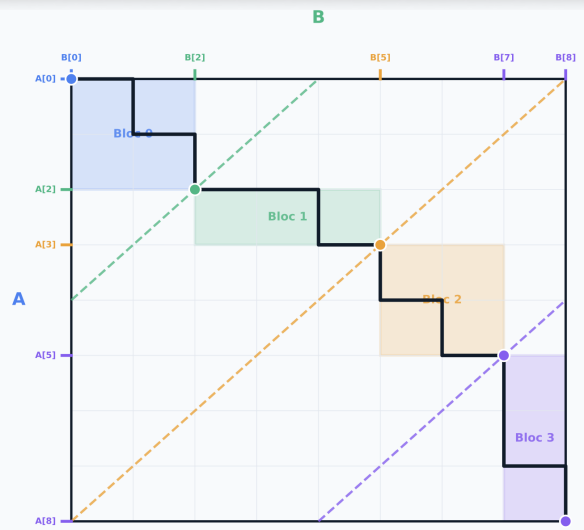
- Entire array loaded into shared memory
- Two buffers (`src` / `dst`) are used
- Buffers are swapped at each pass (no copy)
- All merge passes are performed locally

Complexity

$\log d$ merge passes, each with $O(\log d)$ cost

$\Rightarrow \; O((\log d)^2)$ total time per array

Question 3 : Parallel Merge (Multi-block)



Core idea

Large arrays are partitioned into independent chunks using diagonal intersection (merge path).

Two-step algorithm

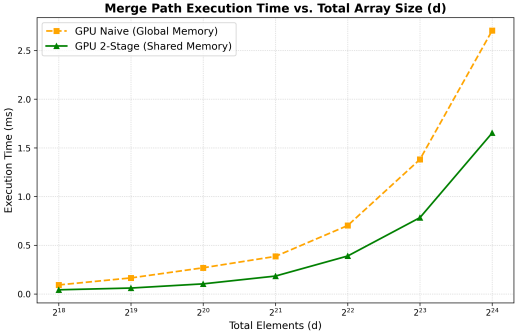
- Partitioning : each block finds its region
- Merging : each block merges its chunk independently

Key advantage

- Perfect load balancing
- No synchronization between blocks
- Fully scalable across GPU

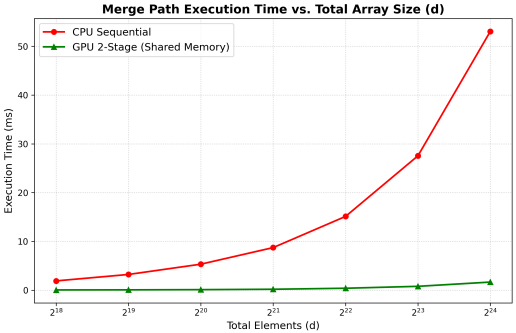
Question 3 : Results

GPU : naive vs optimized



x1.7 faster

CPU vs GPU



x50 faster

References

Research paper

O. Green, R. McColl, D. A. Bader
GPU Merge Path - A GPU Merging Algorithm
26th ACM International Conference on Supercomputing, 2012

Code

`https://github.com/ErwanR123/MergeSort_GPU_Programming`